

Programação e Desenvolvimento de Software I

Arquivos Textuais

Prof. Héctor Azpúrua
(slides adaptados do Prof. Pedro Olmo)

Resumo da Aula Anterior

Arquivos

- Para utilizar um arquivo de dados, é preciso declarar um ponteiro para o tipo **FILE**
- Isso é feito da seguinte forma:

```
FILE *arq;
```

- Com isso, a variável **arq** passa a representar, dentro do programa, um arquivo de dados
- É importante observar que um arquivo de dados é reconhecido pelo sistema operacional como uma entidade externa ao programa

Resumo da Aula Anterior

Abrindo arquivos

- Antes que um arquivo possa ser utilizado, é preciso informar ao sistema operacional onde localizar o arquivo e como se pretende usar o arquivo...
 - Esta operação é conhecida como abertura do arquivo e é executada pela função `fopen`

```
arq = fopen("/meus_docs/matrizes.txt", "r");
```

- A função `fopen` tem dois parâmetros do tipo `string`:
 1. O primeiro parâmetro especifica o **nome/localização** do arquivo
 2. O segundo parâmetro especifica o que **deve ser feito** com os dados

Resumo da Aula Anterior

Abrindo arquivos

- O segundo parâmetro da função `fopen` especifica o que **deve ser feito** com os dados
- As possibilidades são:
 - Ler os dados existentes “r”
 - Gravar dados apagando (caso existam) os dados existentes “w”
 - Anexar novos dados aos já existentes “a”
- A instrução abaixo especifica o modo “r”, ou seja, “ler” (“read”) os dados existentes:

`arq = fopen("matrizes.txt", "r");`



Resumo da Aula Anterior

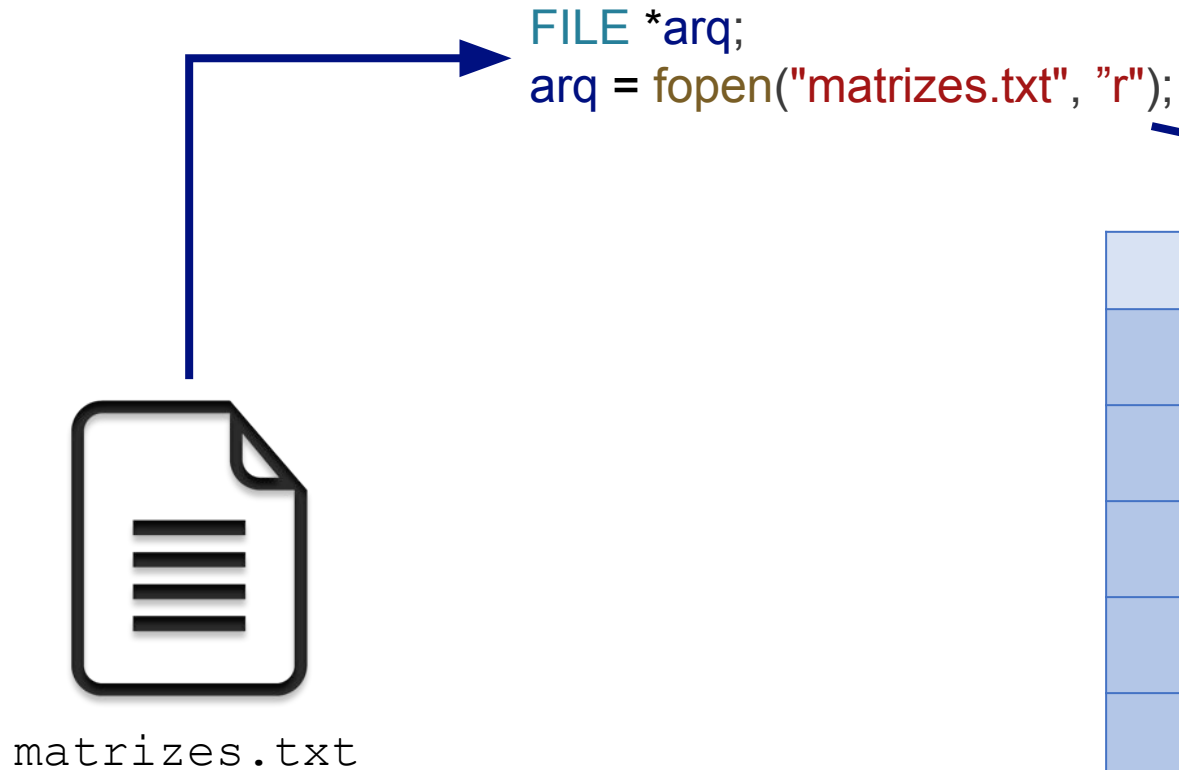
Modos de abertura de arquivos

- A tabela a seguir mostra os possíveis modos de abertura de um arquivo:

Modo	Significado
"r"	Abre um arquivo existente para leitura de dados; se o arquivo não existir, irá ocorrer um erro
"w"	Abre um novo arquivo para gravação de dados; se o arquivo já existir, a gravação irá sobrescrever os dados existentes
"a"	Abre um arquivo para operações de anexação de dados; se o arquivo não existir, será criado um novo arquivo

Resumo da Aula Anterior

Abrindo um arquivo com `fopen`



Endereço	Variável	Conteúdo
0xFF12	arq.campo1	0xFFA4
0xFF13	arq.campo2	1024
0xFF14	arq.campo3	0
0xFF15	arq.campo4	0xFFBB
...	...	...
0xFF (n)	arq.campo (n)	48

`arq == 0xFF12`

Resumo da Aula Anterior

Usando `fscanf`

```
int fscanf(FILE *stream, const char *format, ...);
```

- **FILE *stream**

- Ponteiro para o objeto FILE que identifica o fluxo de entrada do qual os dados devem ser lidos

- **const char *format**

- String de formato (similar ao printf contend o formato dos dados a serem lidos)

- **...**

- Número variável de argumentos adicionais que apontam para os locais onde os valores analisados serão armazenados (ponteiros de variaveis)

- A função `fscanf` **retorna** um valor inteiro que representa o número de itens de entrada correspondidos e atribuídos com sucesso

- Se a entrada terminar antes da primeira falha de correspondência ou se ocorrer um erro, EOF (End Of File) será retornado

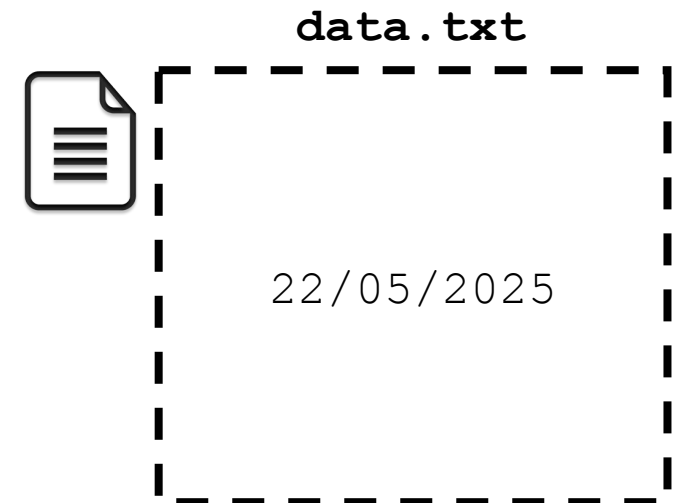
Resumo da Aula Anterior

Usando `fprintf` para escrever num arquivo

- Similar a sintaxe do `fscanf`, podemos usar `fprintf` para escrever num arquivo usando formatadores:

```
#include <stdio.h>
```

```
int main() {  
    FILE *file = fopen("data.txt", "w");  
    int ano = 2025;  
    fprintf(file, "22/05/%d\n", ano);  
    fclose(file);  
}
```



Resumo da Aula Anterior

Fechando arquivos

- Da mesma forma que um programa deve abrir um arquivo antes de usá-lo, **deve também fechar o arquivo** quando não precisar mais dele
 - Arquivos abertos para escritura **estão bloqueados pelo sistema operacional!**
- Na solução do Problema I, o fechamento do arquivo é feito logo após a leitura das duas matrizes:

`fclose(arq);`

- O fechamento de um arquivo instrui o sistema operacional a:
 - Esvaziar os buffers associados ao arquivo
 - Liberar os recursos do sistema que o arquivo consumiu

Arquivos

Problema 2

- Enunciado:
 - Considere que uma matriz de jogadas de dados é gerada aleatoriamente
 - Implemente um programa que, dependendo da escolha do usuário, simplesmente exiba a matriz ou, então, escreva a matriz gerada em um arquivo
- Como fazer com o que conhecemos até agora?

Arquivos

Problema 2

O que esta acontecendo aqui?



stdout é o dispositivo padrão de saída: o vídeo!

Note que este dispositivo é também tratado como um **arquivo**

```
int main() {
    srand((unsigned)time(NULL));
    int n = 3;
    int matriz[n][n];
    for (int i = 0; i < n; i++)
        for (int j = 0; j < n; j++)
            matriz[i][j] = randomInt(1, 6);

    char op = "";
    FILE *arq = NULL;
    do {
        printf("(E)xibir ou (G)gravar a matriz?:");
        op = toupper(getchar());
        if (op == 'E') {
            arq = stdout; // Exibir na tela
        } else if (op == 'G') {
            arq = fopen("matriz_random.txt", "w");
        } else {
            printf("Opção inválida. Tente novamente.\n");
            continue;
        }
    } while (op != 'E' && op != 'G');

    fprintf(arq, "Matriz:\n");
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++)
            fprintf(arq, "%d ", matriz[i][j]);
        fprintf(arq, "\n");
    }
    return 0;
}
```

Dispositivos padrões de entrada e saída

- Dispositivos padrões de entrada e saída de dados (ou também chamados de *streams*) são reconhecidos como arquivos pela linguagem C
 - Eles são designados por constantes predefinidas:

Constante	Significado	Dispositivo
<code>stdin</code>	Dispositivo padrão de entrada	Teclado
<code>stdout</code>	Dispositivo padrão de saída	Vídeo
<code>stderr</code>	Dispositivo padrão de erro	Vídeo
<code>stdaux</code>	Dispositivo padrão auxiliar	Porta serial
<code>stdprn</code>	Dispositivo padrão de impressão	Porta paralela

Dispositivos padrões de entrada e saída

- O que esta acontecendo no **Problema 2?**
 - A matriz de dados é gravada no arquivo representado pelo ponteiro `arq`
 - Dependendo da escolha:
 - o arquivo será o dispositivo padrão de saída (**stdout**) ou
 - o arquivo `matriz_random.txt`
 - A gravação dos dados é feita pela função `fprintf`:

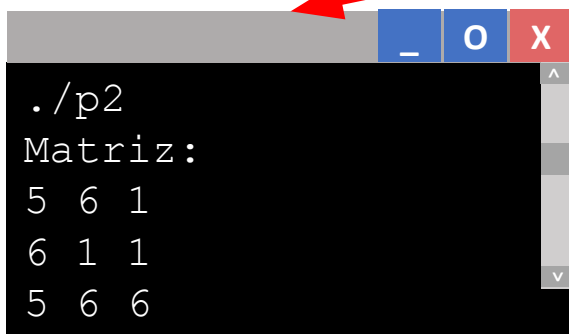
```
fprintf(arq, "Matriz:\n");  
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++)  
        fprintf(arq, "%d ", matriz[i][j]);  
    fprintf(arq, "\n");  
}
```

Dispositivos padrões de entrada e saída

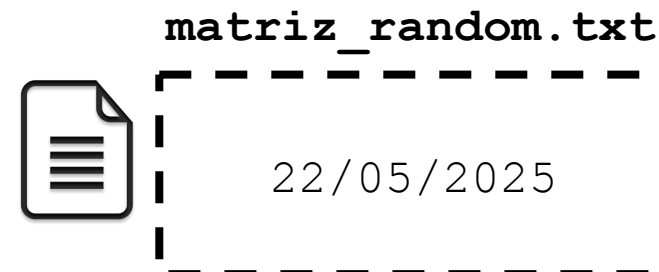
- Basicamente...

- O mesmo trecho de Código pode escrever tanto para o **terminal** como para um **arquivo!**

```
fprintf(arq, "Matriz:\n");  
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++)  
        fprintf(arq, "%d ", matriz[i][j]);  
    fprintf(arq, "\n");  
}
```



A terminal window with a black background and white text. The prompt is `./p2`. The output is `Matriz:` followed by a 3x3 matrix of numbers: `5 6 1`, `6 1 1`, and `5 6 6` on separate lines. The window has a title bar with a minus sign, a maximize button, and a close button.



Dispositivos padrões de entrada e saída

- As funções `fscanf` e `fprintf` são generalizações das funções `scanf` e `printf`
 - Podendo ser usadas para **quaisquer tipos de arquivos**
- Já as funções `scanf` e `printf` só podem ser usadas para os dispositivos padrões (`stdin` e `stdout`)

`scanf("...", ...);` □ □ `fscanf(stdin, "...", ...);`

`printf("...");` □ □ `fprintf(stdout, "...");`

Arquivos

Problema 3

- Enunciado:
 - Um professor armazena em um arquivo as seguintes informações sobre seus alunos:
 - número (`int`)
 - nome (`string`)
 - notas de duas avaliações (`float`)
 - Implemente um programa para listar o conteúdo deste arquivo e exibir a média das notas das avaliações
- Como fazer com o que conhecemos até agora?

Arquivos

Problema 3

- Como podemos?
 - Validar que abrimos o arquivo?

```
if (arq == NULL) {  
    perror("Erro ao abrir o arquivo");  
    return 1;  
}
```

- Falta algo com relação ao arquivo que acabamos de abrir? 🤔
 - Precisamos **fechar** ele!

```
fclose(arq);
```

```
#include <ctype.h>  
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
#define MAX_NOTAS 100
```

```
int main() {  
    FILE *arq;  
    arq = fopen("alunos.txt", "r");  
  
    char buff[MAX_NOTAS];  
    int num;  
    char *nome;  
    float nota1, nota2, total_qty_notas = 0, total_notas = 0;  
    printf("matricula\t nome\t\t nota1\t nota2\n");  
  
    while (fgets(buff, MAX_NOTAS, arq) != NULL) {  
        num = atoi(strtok(buff, ","));  
        nome = strtok(NULL, ",");  
        nota1 = atof(strtok(NULL, ","));  
        nota2 = atof(strtok(NULL, ","));  
        printf("%d\t\t %s\t\t %.1f\t %.1f\n",  
                num, nome, nota1, nota2);  
  
        total_qty_notas += 2;  
        total_notas += nota1 + nota2;  
    }  
    printf("Media: %.2f\n", total_notas / total_qty_notas);  
    return 0;  
}
```

Leitura de dados não-numéricos

Strings

- Quando um arquivo contém informações não numéricas, a leitura dos dados deve ser feita do seguinte modo:
 1. Ler toda uma linha do arquivo como um **string**
 2. Extrair do string lido **substrings** que correspondem aos dados
 3. Converter cada **substring** para o tipo de dado correspondente
- A função `fgets` permite ler toda uma linha de um arquivo como um string

`fgets(buff, MAX, arq);`

Vetor que armazenará o **string**

Tamanho máximo da **string**
(ou termina quando encontra uma
quebra de linha `\n`)

Ponteiro para o **arquivo**

Leitura de dados não-numéricos

Substrings

- Para extrair os **substrings**, é preciso que, no arquivo, os dados estejam separados uns dos outros por um delimitador, diferente de espaço
- Considere, por exemplo, que os dados do arquivo `alunos.txt` estejam separados por vírgulas:

alunos.txt



```
| 1,hector,99,98  
| 2,pedro,80,84  
| 3,rafa,100,99  
| 4,daniel,75,11  
|
```

Leitura de dados não-numéricos

Instrução `strtok`

```
#include <string.h>
```

```
char *strtok(char *string1, const char *string2);
```

- `Strtok` é usado para separar "tokens" numa `string`
- `char *string1`
 - Uma serie de possíveis tokens
- `char *string2`
 - Delimitadores dos tokens na `string1` (**Qualquer** caráter diferente do espaço pode ser usado)
- Quando passado `NULL` no `string1`:
 - O próximo token é lido de copias armazenadas do ultimo parâmetro `string1` não nulo
 - Cada delimitador achado é substituído por um `\0`
 - **Importante:** O valor de `string1` **não é preservado!**
- `Strtok` retorna um ponteiro para o próximo token na sequencia, ou `NULL`

Leitura de dados não-numéricos

Instrução strtok

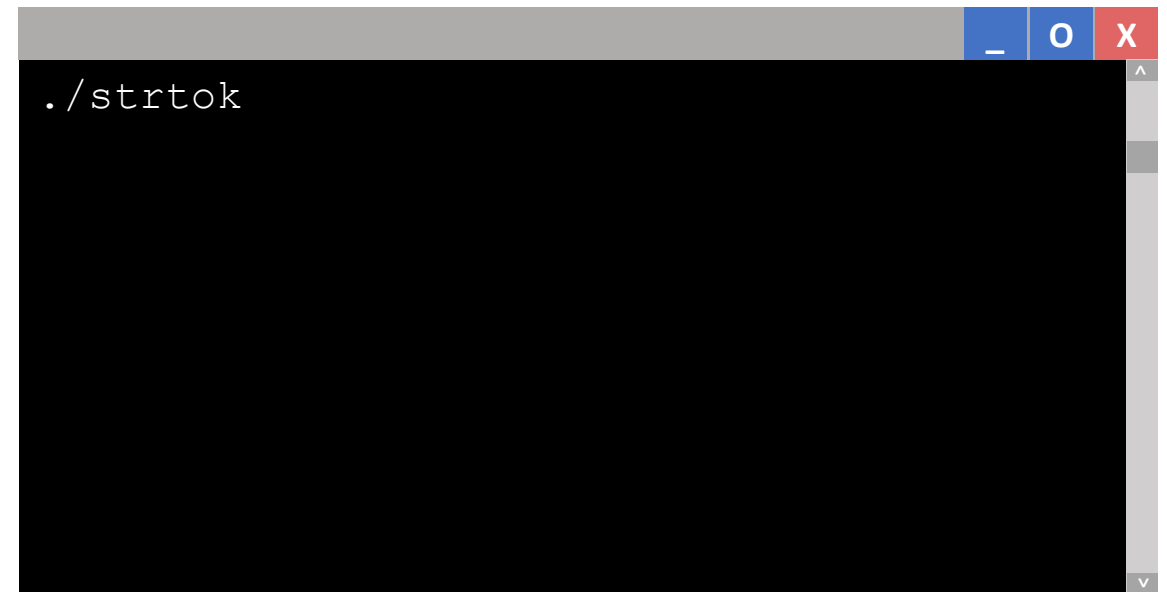
texto	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
	u	m	a		s	t	r	,		t	o	k	e	n	s	,		S	I	M	\0

```
#include <stdio.h>
#include <string.h>
```

```
int main(void) {
    char texto[] = "uma str, tokens,SIM";
    char *token = "";

    printstr(texto);

    token = strtok(texto, ",");
    do {
        printstr(texto);
        printf("\ttoken: %s\n", token);
    } while (token = strtok(NULL, ","));
    printstr(texto);
}
```



```
./strtok
uma str, tokens,SIM
\token: uma str, tokens,SIM
```

Leitura de dados não-numéricos

Instrução strtok

texto	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
	u	m	a		s	t	r	\0		t	o	k	e	n	s	,		S	I	M	\0



```
#include <stdio.h>
#include <string.h>
```

```
int main(void) {
    char texto[] = "uma str, tokens,SIM";
    char *token = "";

    printstr(texto);

    token = strtok(texto, ",");
    do {
        printstr(texto);
        printf("\t\ttoken: %s\n", token);
    } while (token = strtok(NULL, ","));
    printstr(texto);
}
```

```
./strtok
Texto:'uma str, tokens, sim\0'
```

Leitura de dados não-numéricos

Instrução strtok

texto	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
	u	m	a		s	t	r	\0		t	o	k	e	n	s	\0		S	I	M	\0



```
#include <stdio.h>
#include <string.h>
```

```
int main(void) {
    char texto[] = "uma str, tokens, SIM";
    char *token = "";

    printstr(texto);

    token = strtok(texto, ",");
    do {
        printstr(texto);
        printf("\t\ttoken: %s\n", token);
    } while (token = strtok(NULL, ","));
    printstr(texto);
}
```

```
./strtok
Texto: 'uma str, tokens, sim\0'
Texto: 'uma str\0 tokens, sim\0'
      token: 'uma str'
```

Leitura de dados não-numéricos

Instrução `strtok`

- E se meu texto fosse assim?

```
char texto[] = "uma str# tokens#SIM";
```

- Podemos separar ele usando:

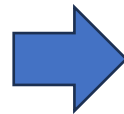
```
token = strtok(texto, "#");
```

- Tokens poder ser misturados também! Cada caráter vira um delimitador

alunos.txt



```
2#Maria da Silva/8.5/4.8  
13#Joao de Almeida/7.5/6.1  
15#Pedro de Souza/5.0/4.4  
21#Jose de Carvalho/7.2/7.1
```



```
str_1 = strtok(buff, "#/");  
str_2 = strtok(NULL, "#/");  
str_3 = strtok(NULL, "#/");  
str_4 = strtok(NULL, "#/");
```


Arquivos

Continuando com o Problema 3...

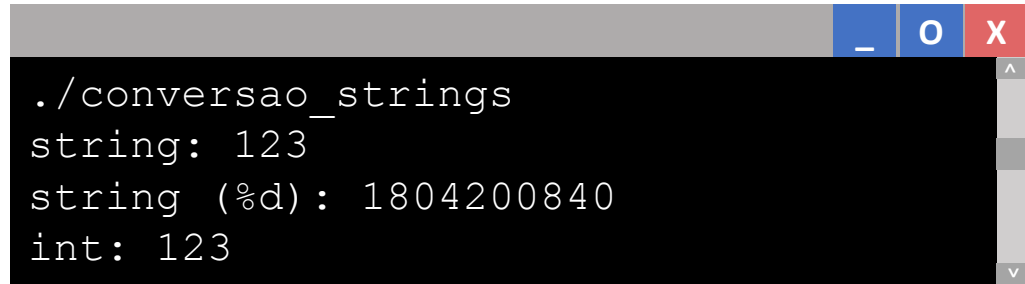
- **Enunciado:**
 - Um professor armazena em um arquivo as seguintes informações sobre seus alunos:
 - número (`int`)
 - nome (`string`)
 - notas de duas avaliações (`float`)
 - Implemente um programa para listar o conteúdo deste arquivo e exibir a média das notas das avaliações
- **Fazendo passo a passo!**

Conversão de strings

Lembrando...

- Converter string para inteiros:
 - `int atoi(const char *string)`
 - `long atol(const char *string)`
 - `long long atoll(const char *string)`
- Converter de string para ponto flutuante
 - `double atof(const char *string)`

```
int main() {  
    char str[] = "123";  
    int novo_int = atoi(str);  
  
    printf("string: %s\n", str);  
    printf("string (%d): %d\n", str);  
    printf("int: %d\n", novo_int);  
  
    return 0;  
}
```



```
./conversao_strings  
string: 123  
string (%d): 1804200840  
int: 123
```

Leitura de dados não-numéricos

Exemplo

- Vamos a ler o arquivo

```
FILE *arq;  
arq = fopen("alunos.txt", "r");  
if (arq == NULL) {  
    perror("Erro ao abrir o arquivo");  
    return 1;  
}
```

alunos.txt



```
1, hector, 99, 98  
2, pedro, 80, 84  
3, rafa, 100, 99  
4, daniel, 75, 11
```

Variáveis

```
#define MAX_NOTAS 100  
FILE *arq;  
char buff[MAX_NOTAS];  
int num;  
char *nome;  
float nota1, nota2, total_qty_notas = 0, total_notas = 0;
```

Leitura de dados não-numéricos

Exemplo

- Ler uma linha do arquivo como uma `string`

```
while (fgets(buff, MAX, arq) != NULL) {  
    ...  
}
```



~~alunos.txt~~



```
1, hector, 99, 98  
2, pedro, 80, 84  
3, rafa, 100, 99  
4, daniel, 75, 11
```

Variáveis

```
#define MAX_NOTAS 100  
FILE *arq;  
char buff[MAX_NOTAS];  
int num;  
char *nome;  
float nota1, nota2, total_qty_notas = 0, total_notas = 0;
```

Leitura de dados não-numéricos

Exemplo

- Extraímos os substrings com `strtok` e convertemos a numero com `atoi`

```
num = atoi(strtok(buff, ","));  
nome = strtok(NULL, ",");  
nota1 = atof(strtok(NULL, ","));  
nota2 = atof(strtok(NULL, ","));
```

buff

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	,	h	e	c	t	o	r	,	9	9	,	9	8	\0							

alunos.txt



```
1, hector, 99, 98  
2, pedro, 80, 84  
3, rafa, 100, 99  
4, daniel, 75, 11
```

Variáveis

```
#define MAX_NOTAS 100  
FILE *arq;  
char buff[MAX_NOTAS];  
int num;  
char *nome;  
float nota1, nota2, total_qty_notas = 0, total_notas = 0;
```

Leitura de dados não-numéricos

Exemplo

- Extraímos os substrings com `strtok` e convertemos a numero com `atoi`

```
num = 1;  
nome = strtok(NULL, ",");  
nota1 = atof(strtok(NULL, ","));  
nota2 = atof(strtok(NULL, ","));
```

buff	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
	1	\0	h	e	c	t	o	r	,	9	9	,	9	8	\0							

alunos.txt



```
1, hector, 99, 98  
2, pedro, 80, 84  
3, rafa, 100, 99  
4, daniel, 75, 11
```

Variáveis

```
#define MAX_NOTAS 100  
FILE *arq;  
char buff[MAX_NOTAS];  
int num;  
char *nome;  
float nota1, nota2, total_qty_notas = 0, total_notas = 0;
```

Leitura de dados não-numéricos

Exemplo

- Extraímos os substrings com `strtok` e convertemos a numero com `atoi`

```
num = 1;  
nome = "hector";  
nota1 = atof(strtok(NULL, ","));  
nota2 = atof(strtok(NULL, ","));
```

buff	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
	1	\0	h	e	c	t	o	r	\0	9	9	,	9	8	\0							

alunos.txt



```
1, hector, 99, 98  
2, pedro, 80, 84  
3, rafa, 100, 99  
4, daniel, 75, 11
```

↑
variáveis

```
#define MAX_NOTAS 100  
FILE *arq;  
char buff[MAX_NOTAS];  
int num;  
char *nome;  
float nota1, nota2, total_qty_notas = 0, total_notas = 0;
```

Leitura de dados não-numéricos

Exemplo

- Extraímos os substrings com `strtok` e convertemos a numero com `atoi`

```
num = 1;  
nome = "hector";  
nota1 = 99;  
nota2 = atof(strtok(NULL, ","));
```

buff

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
1	\0	h	e	c	t	o	r	\0	9	9	\0	9	8	\0							

alunos.txt



```
1,hector,99,98  
2,pedro,80,84  
3,rafa,100,99  
4,daniel,75,11
```

Variáveis

```
#define MAX_NOTAS 100  
FILE *arq;  
char buff[MAX_NOTAS];  
int num;  
char *nome;  
float nota1, nota2, total_qty_notas = 0, total_notas = 0;
```


Leitura de dados não-numéricos

Exemplo

- Extraímos os substrings com `strtok` e convertemos a numero com `atoi`

```
num = 1;  
nome = "hector";  
nota1 = 99;  
nota2 = 98;
```

buff	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21
	1	\0	h	e	c	t	o	r	\0	9	9	\0	9	8	\0							

alunos.txt



```
1,hector,99,98  
2,pedro,80,84  
3,rafa,100,99  
4,daniel,75,11
```

Variáveis

```
#define MAX_NOTAS 100  
FILE *arq;  
char buff[MAX_NOTAS];  
int num;  
char *nome;  
float nota1, nota2, total_qty_notas = 0, total_notas = 0;
```



Arquivos

Problema 4

- Enunciado:
 - Modifique o programa anterior de modo a permitir que novos alunos sejam incluídos no arquivo
- Como fazer com o que conhecemos até agora?

Arquivos

Problema 4

```
#define MAX_SIZE 100
```

```
int main() {  
    FILE *arq;  
    arq = fopen("alunos.txt", "a");  
  
    char buff[MAX_SIZE];  
    char *token = "";  
    int num;  
    char *nome;  
    float nota1, nota2;  
  
    printf("Digite a matricula, nome, nota1 e nota2 do aluno:\n");  
    fgets(buff, MAX_SIZE, stdin);  
  
    num = atoi(strtok(buff, ","));  
    nome = strtok(NULL, ",");  
    nota1 = atof(strtok(NULL, ","));  
    nota2 = atof(strtok(NULL, ","));  
    fprintf(arq, "%d,%s,%4.1f,%4.1f\n", num, nome, nota1, nota2);  
    fclose(arq);  
}
```

O que aconteceria se usamos "w" aqui? 🤔



Perguntas?

- E-mail:
 - hector@dcc.ufmg.br
- Material da disciplina:
 - <https://pedroolmo.github.io/teaching/pdsI.html>
- Github:
 - <https://github.com/h3ct0r>



Héctor Azpúrua
h3ct0r